

DOM Manipulation - Code Based Notes

1. insertAdjacentElement(position, element)

The `insertAdjacentElement()` method inserts a given element node at a specified position relative to the element it is invoked upon.

Syntax

```
element.insertAdjacentElement(position, elementToInsert)
```

Positions

- `beforebegin` - Before the target element itself
- `afterbegin` - Just inside the target element, before its first child
- `beforeend` - Just inside the target element, after its last child
- `afterend` - After the target element itself

Visual Representation

```
<!-- beforebegin -->  
<div id="target">  
  <!-- afterbegin -->  
  <p>existing content</p>  
  <!-- beforeend -->  
</div>  
<!-- afterend -->
```

Detailed Code Example

```

//! insertAdjacentElement(position, element)

const div = document.getElementById("root");

// Create paragraph elements
const p1 = document.createElement("p");
p1.textContent = "Paragraph - 1";

const p3 = document.createElement("p");
p3.textContent = "Paragraph - 3";

// Insert elements inside the div
div.insertAdjacentElement("afterbegin", p1); // Adds as first child
div.insertAdjacentElement("beforeend", p3); // Adds as last child

// Create heading elements for outside positioning
const t1 = document.createElement("h2");
const t2 = document.createElement("h2");

// Set content and styles
t1.textContent = "Start";
t2.textContent = "End";
t1.style.color = "green";
t2.style.color = "red";

// Insert elements outside the div
div.insertAdjacentElement("beforebegin", t1); // Adds before the div
div.insertAdjacentElement("afterend", t2); // Adds after the div

/*
Result structure:
<h2 style="color: green">Start</h2>      <!-- beforebegin -->
<div id="root">
  <p>Paragraph - 1</p>                  <!-- afterbegin -->
  <p>existing content</p>
  <p>Paragraph - 3</p>                  <!-- beforeend -->
</div>
<h2 style="color: red">End</h2>         <!-- afterend -->
*/

```

2. Attribute Methods

setAttribute(attributeName, attributeValue)

Sets the value of an attribute on the specified element.

getAttribute(attributeName)

Returns the value of a specified attribute on the element.

removeAttribute(attributeName)

Removes the attribute with the specified name from the element.

Detailed Code Example

```

//! setAttribute();
//! getAttribute();
//! removeAttribute();

const div = document.querySelector("div");

// Get the current class attribute
const x = div.getAttribute("class");
console.log("Current class:", x); // Shows existing classes

// Add new class while keeping existing ones
div.setAttribute("class", `${x} justify-center`);

// Remove the id attribute completely
div.removeAttribute("id");

// Check the updated class attribute
const updatedClass = div.getAttribute("class");
console.log("Updated class:", updatedClass);

/*
Key Points:
- getAttribute() returns the attribute value or null if not found
- setAttribute() replaces the entire attribute value
- To preserve existing values, concatenate with template literals
- removeAttribute() completely removes the attribute
*/

```

Important Notes:

- If the attribute already exists, `setAttribute()` replaces the previous value
- To preserve existing values, first get the current value and concatenate
- `getAttribute()` returns `null` if the attribute doesn't exist.

3. Content Manipulation Properties

innerHTML

Gets or sets the HTML markup contained within the element.

outerHTML

Gets or sets the HTML markup of the element including the element itself.

innerText

Gets or sets the text content of the element and its descendants.

Detailed Code Example

```

/*
- innerHTML
- outerHTML
- innerText
- textContent
*/

const div = document.querySelector("#root");

// innerHTML - adds HTML content inside the element
div.innerHTML = "<h1>Hello world</h1>"; // Replaces all content inside div

/*
What happens:
Before: <div id="root"><p>old content</p></div>
After:  <div id="root"><h1>Hello world</h1></div>
*/

// outerHTML - replaces the entire element
// div.outerHTML = "<section><h1>New Section</h1></section>";

/*
What would happen with outerHTML:
Before: <div id="root">content</div>
After:  <section><h1>New Section</h1></section>
*/

// innerText - sets only plain text (no HTML)
// div.innerText = "Just plain text";

/*
Differences:
- innerHTML: Parses HTML tags, adds inside element
- outerHTML: Replaces entire element structure
- innerText: Only plain text, ignores HTML tags
- textContent: Similar to innerText but includes hidden elements
*/

```

4. DOM Traversal

Detailed Code Example

```
// traversing an element

const div = document.querySelector("#root");

// Parent element
console.log(div.parentElement);
// Output: Shows the parent element (usually <body>)

// Current element
console.log(div);
// Output: Shows the div element itself

// Children collection (only elements, no text nodes)
console.log(div.children); // HTMLCollection with 3 elements
// Output: HTMLCollection(3) [p, p, p]

// All child nodes (includes text nodes like whitespace/line breaks)
console.log(div.childNodes); // NodeList with 7 nodes (includes empty spaces)
// Output: NodeList(7) [text, p, text, p, text, p, text]

// First and last element children
console.log(div.firstElementChild);
// Output: <p>Paragraph Tag-1</p>

console.log(div.lastElementChild);
// Output: <p>Paragraph Tag-3</p>

// Sibling navigation
console.log(div.firstElementChild.nextElementSibling); // Second paragraph (p2)
// Output: <p>Paragraph Tag-2</p>

console.log(div.lastElementChild.previousElementSibling); // Second paragraph (p2)
// Output: <p>Paragraph Tag-2</p>

/*
Key Differences:
- children: Only element nodes (HTMLCollection) - Count: 3
- childNodes: All nodes including text/whitespace (NodeList) - Count: 7

Why childNodes has 7 items:
1. text node (whitespace before first <p>)
2. <p>Paragraph Tag-1</p>
3. text node (whitespace between p tags)
4. <p>Paragraph Tag-2</p>
5. text node (whitespace between p tags)
6. <p>Paragraph Tag-3</p>
7. text node (whitespace after last <p>) */
```

5. Element Removal

remove()

Removes the element from the DOM (called on the element itself).

removeChild(childElement)

Removes a specified child element from the parent (called on parent element).

Detailed Code Example

```

/**
 * ! remove()
 * ! removeChild()
 */

const div = document.querySelector("#root");

// Method 1: Direct removal using remove()
// div.children[2].remove(); // This would remove the third child element

/*
How remove() works:
- Called directly on the element you want to remove
- Element removes itself from its parent
- Simple and direct approach
*/

// Method 2: Parent removes child using removeChild()
const p2 = document.querySelector("#root :nth-of-type(2)"); // Selects 2nd paragraph
div.removeChild(p2); // Parent div removes the p2 element

/*
How removeChild() works:
- Called on the parent element
- Requires reference to the child element
- Parent explicitly removes the specified child
- Returns the removed element (can be stored in variable)

Selector explanation:
"#root :nth-of-type(2)" means:
- Inside element with id="root"
- Find the 2nd element of its type (2nd <p> tag)
*/

// Alternative removal methods:
// Remove by index
// div.children[1].remove();

// Remove first child
// div.firstElementChild.remove();

// Remove last child
// div.lastElementChild.remove();

/*
Key Differences:
remove():
- element.remove()
- Called on element itself
- Modern method (ES6+)
- Simpler syntax

removeChild():
- parent.removeChild(child)
- Called on parent element
- Older method (works in all browsers)
- Returns removed element
- Need reference to both parent and child
*/

```

6. Best Practices and Tips

Performance Considerations

- Use `children` instead of `childNodes` when you only need element nodes
- Cache DOM queries in variables to avoid repeated selections
- Use `DocumentFragment` for multiple element insertions

Error Handling

```
// Always check if elements exist before manipulation
const element = document.querySelector("#myElement");
if (element) {
  element.setAttribute("class", "new-class");
}
```

Common Patterns

```
// Creating and inserting multiple elements
const fragment = document.createDocumentFragment();
for (let i = 0; i < 5; i++) {
  const p = document.createElement("p");
  p.textContent = `Paragraph ${i + 1}`;
  fragment.appendChild(p);
}
document.getElementById("root").appendChild(fragment);
```